

Laboratory Report: Lab 4 List

Zofia Warzycha, 280971

April 2026

1 Task 1: Smartwatch System

1.1 Task Description

The goal of this task was to design and implement a **Smartwatch** system in Java using abstract classes and interfaces. The system models different types of smartwatches from various manufacturers.

The program should:

- implement an abstract class **Smartwatch** with fields for serial number, paired phone number, battery level, connection statuses, activity history, notification history, and a **WatchProducer** enum,
- use a static factory method to validate input before creating objects,
- create two subclasses: **AndroidWatch** and **AppleWatch**,
- implement five interfaces: **Trackable**, **Connectable**, **Payable**, **Chargeable**, and **Notifiable**,
- store every completed workout and received notification with a **LocalDateTime** timestamp,
- override **toString()** to display the current watch status.

If the serial number is empty or the phone number is not a 9-digit string, the factory method should throw an **IllegalArgumentException**. If the producer does not match the watch type, an exception is also thrown.

1.2 Source Code

Interfaces and Enum

I started by defining the **WatchProducer** enum and all five interfaces. The enum lists all supported manufacturers – **APPLE** is separate from the Android-based ones because it is used later to validate which watch type can be created with which producer.

Each interface was kept as small as possible, containing only the methods that belong to that specific responsibility. For example, **Trackable** only deals with starting and stopping workouts, and **Notifiable** only handles receiving and displaying notifications. This way each interface has a single, clear purpose.

Connectable is the only interface implemented directly in the abstract **Smartwatch** class, because WiFi and Bluetooth connection logic is the same for all watch types. **Payable** and **Chargeable** are implemented only in the subclasses, because Android and Apple use completely different payment and charging systems.

```
1 public enum WatchProducer {
2     APPLE, SAMSUNG, GARMIN, HUAWEI, XIAOMI
3 }
4
5 public interface Trackable {
6     void startWorkout(String workoutType);
7     void stopWorkout();
8 }
9
10 public interface Connectable {
11     void connectWifi();
12     void disconnectWifi();
13     void connectBluetooth();
14     void disconnectBluetooth();
15 }
16
17 public interface Payable {
18     void makePayment(double amount);
19 }
20
21 public interface Chargeable {
22     void charge(int percent);
23 }
24
25 public interface Notifiable {
26     void receiveNotification(String message);
27     void showNotificationHistory();
28 }
```

Listing 1: WatchProducer enum and interfaces

Abstract Smartwatch Class

The **Smartwatch** class is declared **abstract** because it represents a general concept – a smartwatch – that should never be instantiated on its own. It holds all the fields that are shared between **AndroidWatch** and **AppleWatch**, so I do not have to repeat them in both subclasses.

The constructor is **protected** so it can only be called from inside the class hierarchy. The battery level starts at 60% by default, and both connection flags start as **false**, which matches the expected initial output shown in the task description. I made **batteryLevel** **protected** instead of **private** so that the subclasses can update it directly inside their **charge()** methods without needing an extra setter in the parent class.

The static **validateInput()** method handles the two basic checks – empty serial number and incorrect phone number format – and throws an **IllegalArgumentException** if

either check fails. I put it in the abstract class so both `AndroidWatch.create()` and `AppleWatch.create()` can call it without duplicating the validation code.

The listing below shows the key parts of the class. The implementation of `Connectable` (the four connect/disconnect methods) is standard and not included here to keep the listing shorter.

```
1 import java.util.List;
2 import java.util.ArrayList;
3 import java.time.LocalDateTime;
4
5 public abstract class Smartwatch implements Connectable, Notifiable
6     , Trackable {
7
8     private String serialNumber;
9     private String pairedPhoneNumber;
10    private List<String> activityHistory;
11    protected int batteryLevel;
12    private boolean bluetoothConnected;
13    private boolean wifiConnected;
14    private List<String> notificationHistory;
15    private WatchProducer producer;
16    private String currentWorkout;
17
18    protected Smartwatch(String serialNumber, String
19        pairedPhoneNumber,
20        WatchProducer producer) {
21        this.serialNumber = serialNumber;
22        this.pairedPhoneNumber = pairedPhoneNumber;
23        this.producer = producer;
24        this.batteryLevel = 60;
25        this.bluetoothConnected = false;
26        this.wifiConnected = false;
27        this.activityHistory = new ArrayList<>();
28        this.notificationHistory = new ArrayList<>();
29    }
30
31    public static void validateInput(String serialNumber,
32        String pairedPhoneNumber) {
33        if (serialNumber == null || serialNumber.isEmpty()) {
34            throw new IllegalArgumentException(
35                "Serial number cannot be empty.");
36        }
37        if (pairedPhoneNumber == null
38            || !pairedPhoneNumber.matches("\\d{9}")) {
39            throw new IllegalArgumentException(
40                "Paired phone number must be a 9-digit number.");
41        }
42    }
43
44    @Override
45    public void startWorkout(String workoutType) {
```

```

44         this.currentWorkout = workoutType;
45         System.out.println("Workout started: " + workoutType);
46     }
47
48     @Override
49     public void stopWorkout() {
50         if (currentWorkout == null) {
51             System.out.println("No workout in progress.");
52             return;
53         }
54         String entry = "[" + LocalDateTime.now()
55             + "] Workout completed: " + currentWorkout;
56         activityHistory.add(entry);
57         System.out.println("Workout stopped.");
58         currentWorkout = null;
59     }
60
61     @Override
62     public void receiveNotification(String message) {
63         String entry = "[" + LocalDateTime.now()
64             + "] Notification: " + message;
65         notificationHistory.add(entry);
66         System.out.println("Notification received: " + message);
67     }
68
69     @Override
70     public void showNotificationHistory() {
71         if (notificationHistory.isEmpty()) {
72             System.out.println("No notifications.");
73             return;
74         }
75         for (String n : notificationHistory) {
76             System.out.println(n);
77         }
78     }
79
80     public WatchProducer getProducer() {
81         return producer;
82     }
83
84     public int getBatteryLevel() {
85         return batteryLevel;
86     }
87
88     public String getSerialNumber() {
89         return serialNumber;
90     }
91
92     public abstract String getSystem();
93
94     @Override

```

```

95     public String toString() {
96         return "Watch Info:\n" +
97             "Producer: " + producer + "\n" +
98             "System: " + getSystem() + "\n" +
99             "Paired Number: " + pairedPhoneNumber + "\n" +
100            "Serial Number: " + serialNumber + "\n" +
101            "Battery Level: " + batteryLevel + "%\n" +
102            "WiFi: " + (wifiConnected ? "Connected" : "Not
                connected") + "\n" +
103            "Bluetooth: " + (bluetoothConnected ? "Connected"
104                               : "Not connected")
                + "\n" +
105            "Tracked Activities: " + activityHistory.size() + "\n"
                +
106            "Notifications: " + notificationHistory.size();
107     }
108 }

```

Listing 2: Abstract Smartwatch class – key methods

The `stopWorkout()` method first checks if a workout is actually in progress before saving anything. If `currentWorkout` is `null` it just prints a message and returns early. Otherwise it builds a string with the current timestamp from `LocalDateTime.now()` and adds it to `activityHistory` – this covers the extra task requirement, since every completed workout is automatically recorded with the date and time. The `receiveNotification()` method works the same way.

The `getSystem()` method is declared `abstract` because each watch type runs a different operating system – `AndroidWatch` returns "Wear OS" and `AppleWatch` returns "watchOS". Since the parent class cannot know this, the subclasses are forced to provide their own implementation. The getters like `getProducer()` are needed by the subclasses – for example `charge()` calls `getProducer()` to print which manufacturer's watch is charging.

AndroidWatch and AppleWatch Subclasses

Both subclasses extend `Smartwatch` and additionally implement `Payable` and `Chargeable`. Their constructors are `private` so objects can only be created through the static `create()` factory method, which runs validation first.

In `AndroidWatch.create()`, after calling `validateInput()`, I check whether the given producer is one of the four valid Android manufacturers using a chain of `!=` conditions. If none of them match, an `IllegalArgumentException` is thrown. This prevents creating, for example, an Android watch with the APPLE producer.

The `charge()` method in both subclasses adds the given percentage directly to `batteryLevel` and then caps it at 100 with a simple `if` statement, so the battery can never go above 100%. The two classes differ in the charging message and in the payment method name – magnetic charger and Google Pay for Android, MagSafe and Apple Pay for Apple.

```

1 public class AndroidWatch extends Smartwatch implements Payable,
    Chargeable {

```

```

2
3 private AndroidWatch(String serialNumber, String
    pairedPhoneNumber,
4         WatchProducer producer) {
5     super(serialNumber, pairedPhoneNumber, producer);
6 }
7
8 public static AndroidWatch create(String serialNumber,
9         String pairedPhoneNumber,
10        WatchProducer producer) {
11     validateInput(serialNumber, pairedPhoneNumber);
12
13     if (producer != WatchProducer.SAMSUNG && producer !=
        WatchProducer.GARMIN
14         && producer != WatchProducer.HUAWEI
15         && producer != WatchProducer.XIAOMI) {
16         throw new IllegalArgumentException(
17             "Producer " + producer + " is not valid for
                AndroidWatch.");
18     }
19
20     return new AndroidWatch(serialNumber, pairedPhoneNumber,
        producer);
21 }
22
23 @Override
24 public void charge(int percent) {
25     System.out.println(getProducer()
26         + " smartwatch charging via magnetic charger.");
27     batteryLevel += percent;
28     if (batteryLevel > 100) {
29         batteryLevel = 100;
30     }
31     System.out.println("Charging... Battery is now at "
32         + batteryLevel + "%.");
33 }
34
35 @Override
36 public void makePayment(double amount) {
37     System.out.println(getProducer()
38         + " smartwatch making payment of "
39         + amount + " via Google Pay.");
40 }
41
42 @Override
43 public String getSystem() { return "Wear OS"; }
44 }

```

Listing 3: AndroidWatch subclass

```

1 public class AppleWatch extends Smartwatch implements Payable,
    Chargeable {

```

```

2
3     private AppleWatch(String serialNumber, String
4         pairedPhoneNumber) {
5         super(serialNumber, pairedPhoneNumber, WatchProducer.APPLE)
6         ;
7     }
8
9     public static AppleWatch create(String serialNumber,
10        String pairedPhoneNumber,
11        WatchProducer producer) {
12        validateInput(serialNumber, pairedPhoneNumber);
13
14        if (producer != WatchProducer.APPLE) {
15            throw new IllegalArgumentException(
16                "AppleWatch must have producer APPLE, got: " +
17                producer);
18        }
19        return new AppleWatch(serialNumber, pairedPhoneNumber);
20    }
21
22    @Override
23    public void charge(int percent) {
24        System.out.println("Apple Watch charging via MagSafe
25            charger.");
26        batteryLevel += percent;
27        if (batteryLevel > 100) {
28            batteryLevel = 100;
29        }
30        System.out.println("Charging... Battery is now at "
31            + batteryLevel + "%.");
32    }
33
34    @Override
35    public void makePayment(double amount) {
36        System.out.println("Apple Watch making payment of "
37            + amount + " via Apple Pay.");
38    }
39
40    @Override
41    public String getSystem() { return "watchOS"; }
42 }

```

Listing 4: AppleWatch subclass

Tester Class

In the tester I created a `List<Smartwatch>` to store all watches together, which is possible because both subclasses extend `Smartwatch`. I added two `AndroidWatch` objects and one `AppleWatch` to the list. Then I tested all the main methods on the Samsung watch – charging, connecting WiFi and Bluetooth, starting and stopping a workout, and receiving a notification – and printed the watch status before and after the operations to verify the

output matches the expected format from the task.

At the end I also tested two invalid cases inside try-catch blocks: creating an AppleWatch with a non-Apple producer, and creating an AndroidWatch with an incorrect phone number. Both throw an IllegalArgumentException with an appropriate message, which is caught and printed.

```
1 import java.util.List;
2 import java.util.ArrayList;
3
4 public class SmartwatchTester {
5
6     public static void main(String[] args) {
7
8         List<Smartwatch> watches = new ArrayList<>();
9
10        AndroidWatch samsungWatch = AndroidWatch.create(
11            "SWA-2026-001", "123456789", WatchProducer.SAMSUNG);
12        AndroidWatch garminWatch = AndroidWatch.create(
13            "GWA-2026-002", "987654321", WatchProducer.GARMIN);
14        AppleWatch appleWatch = AppleWatch.create(
15            "AWA-2026-003", "111222333", WatchProducer.APPLE);
16
17        watches.add(samsungWatch);
18        watches.add(garminWatch);
19        watches.add(appleWatch);
20
21        System.out.println(samsungWatch);
22        System.out.println();
23
24        samsungWatch.charge(15);
25        samsungWatch.connectWifi();
26        samsungWatch.connectBluetooth();
27        samsungWatch.startWorkout("Running");
28        samsungWatch.stopWorkout();
29        samsungWatch.receiveNotification("Meeting at 14:00");
30
31        System.out.println();
32        System.out.println(samsungWatch);
33
34        // Test invalid producer -- should throw exception
35        try {
36            AppleWatch invalid = AppleWatch.create(
37                "BAD-001", "000000000", WatchProducer.SAMSUNG);
38        } catch (IllegalArgumentException e) {
39            System.out.println("Exception caught: " + e.getMessage());
40        }
41
42        // Test invalid phone number
43        try {
44            AndroidWatch bad = AndroidWatch.create(
```



```

45         "BAD-002", "12345", WatchProducer.SAMSUNG);
46     } catch (IllegalArgumentException e) {
47         System.out.println("Exception caught: " + e.getMessage
48             ());
49     }
50 }

```

Listing 5: SmartwatchTester class

1.3 Console Output

```

"C:\Program Files\Java\jdk-25\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ
Watch Info:
Producer: SAMSUNG
System: Wear OS
Paired Number: 123456789
Serial Number: SWA-2026-001
Battery Level: 60%
WiFi: Not connected
Bluetooth: Not connected
Tracked Activities: 0
Notifications: 0

SAMSUNG smartwatch charging via magnetic charger.
Charging... Battery is now at 75%.
SAMSUNG smartwatch connected to WiFi.
SAMSUNG smartwatch connected to Bluetooth.
Workout started: Running
Workout stopped.
Notification received: Meeting at 14:00

Watch Info:
Producer: SAMSUNG
System: Wear OS
Paired Number: 123456789
Serial Number: SWA-2026-001
Battery Level: 75%
WiFi: Connected
Bluetooth: Connected
Tracked Activities: 1
Notifications: 1

```

Figure 1: Console output

```
Samsung notification history:
[2026-04-08T10:34:11.135205500] Notification: Meeting at 14:00

Apple notification history:
[2026-04-08T10:34:11.136206600] Notification: Low battery

SAMSUNG smartwatch making payment of 15.5 via Google Pay.

SAMSUNG smartwatch disconnected from WiFi.
SAMSUNG smartwatch disconnected from Bluetooth.

Testing invalid producer for AppleWatch:
Exception caught: AppleWatch must have producer APPLE, got: SAMSUNG

Testing invalid phone number:
Exception caught: Paired phone number must be a 9-digit number.
```

Figure 2: Console output

2 Conclusions

- I learned how to design an abstract class that implements multiple interfaces at once, which reduces code duplication across the subclasses.
- Using a static factory method instead of a public constructor made it possible to validate all input values before creating an object and throw an `IllegalArgumentException` with a clear error message.
- Assigning `Payable` and `Chargeable` only to the subclasses was the right choice, because Android and Apple devices use completely different payment and charging systems.
- I learned that storing timestamps with `LocalDateTime.now()` inside each activity and notification entry is a simple and effective way to implement the extra task without adding extra fields to the class.
- Making `batteryLevel` `protected` instead of `private` allowed the subclasses to update it directly in the `charge()` method without needing an extra helper method in the parent class.